PrimeTest — Large Prime Analysis & Validation Platform

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

Mohammad Rehan (2520030066)
K. Sai Akhil (2520030021)
T. Hasini (2520030102)

Team Number: Team 3

Section: S1

Under the guidance of

Dr. CH Anuradha

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The mini project report entitled "PrimeTest — Large Prime Analysis & Validation Platform" is a record of bonafide work carried out by Mohammad Rehan (2520030066), K. Sai Akhil (2520030021), and T. Hasini (2520030102), submitted in partial fulfillment of the requirements for the course Data Structures and Algorithms -3 (25CS2103E) at K L (Deemed to be) University. The work presented in this report is our own.

K.Sai Akhil — 2520030021

M.Rehan — 2520030066

T.Hasini — 2520030102

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project report entitled "PrimeTest — Large Prime Analysis & Validation Platform" is a bonafide work carried out by Mohammad Rehan (2520030066), K. Sai Akhil (2520030021), and T. Hasini (2520030102), in partial fulfillment of the requirements for the course Data Structures and Algorithms -3 (25CS2103E), during the academic year 2026–2027.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide for the guidance, feedback, and encouragement provided during the development of PrimeTest. We also thank the Department of Computer Science and Engineering and K L (Deemed to be) University for providing the learning environment and resources needed to complete this project. Finally, we thank our classmates, friends, and families for their support.

We are especially thankful to our guide, [Guide Name], [Designation, Department], for valuable suggestions and support throughout the project.

We thank the faculty and staff of the university for their assistance and for supporting our learning in Data Structures and Algorithms.

We are grateful to everyone who helped us directly or indirectly during the completion of this project.

K.Sai Akhil — 2520030021

M.Rehan — 2520030066

T.Hasini — 2520030102

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

PrimeTest is a Java-based terminal application for analyzing the primality of integers, including values larger than the range supported by Java's primitive integer types. The application uses the Miller–Rabin probabilistic primality test as its primary algorithm and, for relatively small inputs, compares the result with a deterministic trial-division method. Java's BigInteger class supports arbitrary-precision integer operations needed for large inputs. The program reports whether an input is composite or probably prime, the number of Miller–Rabin iterations, execution time, an upper bound on the probability of error, and the corresponding confidence percentage. Trial division is skipped for larger inputs because checking every possible divisor up to the square root can be impractical. The project demonstrates the use of modular exponentiation, repeated squaring, randomized witnesses, algorithm analysis, and execution-time measurement. The current review scope is a terminal-based core implementation; a web interface, prime generation, benchmarking extensions, and parallel processing are considered future enhancements. Miller–Rabin is probabilistic, so "probably prime" is used rather than claiming an absolute proof of primality.

Team 3

Team Members:

1. K.Sai Akhil, 2520030021
2. M.Rehan, 2520030066
3. T.Hasini, 2520030102

2. INTRODUCTION

Prime numbers are important in number theory and are widely used in cryptography and computer security. Testing whether a very large integer is prime can be expensive when using a straightforward deterministic method such as trial division. Trial division checks candidate divisors up to the square root of the input, which becomes impractical as the input grows. PrimeTest addresses this challenge through a Java terminal application that uses the Miller–Rabin primality test. Miller–Rabin uses modular exponentiation and randomly selected witnesses to identify composite numbers efficiently. If a witness demonstrates compositeness, the number is composite; if all tested witnesses pass, the number is reported as probably prime with a stated error bound. The application uses BigInteger so it can represent integers beyond the fixed range of int and long. For smaller inputs, PrimeTest also runs trial division to provide a comparison of results and execution times. This report presents the algorithm background, objectives, requirements, implementation modules, sample results, and possible future improvements.

The introduction is meant to engage the reader, providing enough context to appreciate the project's purpose and importance while also leading smoothly into the project details. It usually ends with a brief outline of what the rest of the document will cover.

3. LITERATURE SURVEY

1. Miller–Rabin primality testing. Miller's work established a primality-testing approach related to the extended Riemann hypothesis, and Rabin developed a randomized version whose practical form is widely taught and used. The algorithm repeatedly tests randomly chosen bases and can identify compositeness when a witness fails. Its error probability decreases exponentially as the number of independent rounds increases.

2. Handbook of Applied Cryptography. This reference explains primality testing and the role of probabilistic tests in cryptographic systems. It provides mathematical background for modular arithmetic and primality-testing methods.

3. Java BigInteger documentation. Java's BigInteger class provides arbitrary-precision integer arithmetic, comparisons, bit-length operations, and modular exponentiation. These capabilities make it suitable for implementing Miller–Rabin for integers larger than primitive types can represent.

4. Trial division. Trial division is a simple deterministic baseline: test divisors from 2 up to the square root of the input. It is easy to understand and verify for small numbers, but its running time grows quickly for large inputs.

5. Modular exponentiation. Efficient modular exponentiation computes a^d mod n without constructing the potentially enormous value a^d. It is a core operation in Miller–Rabin and helps make the algorithm practical for large integers.

6. Probabilistic primality testing. Probabilistic algorithms trade a small, quantifiable error

probability for improved performance. Repeating independent rounds lowers the standard Miller–Rabin error bound to at most (1/4)^k for k rounds.

4. Problem Statement and Objectives

Problem Statement: To provide an efficient Java-based method for testing whether large integers are prime or composite.

Problem Description: Traditional trial division becomes impractical for large integers because it may examine many candidate divisors. Prime Test uses Miller–Rabin for efficient testing and compares it with trial division only for smaller inputs.

Objectives:

- Accept an integer from the terminal and validate its primality using Miller–Rabin.
- Use BigInteger to support arbitrary-precision integer inputs.
- Allow the user to choose the number of Miller–Rabin iterations.
- Display the result, iteration count, execution time, error bound, and confidence estimate.
- Compare Miller–Rabin with deterministic trial division for relatively small inputs.
- Avoid running impractically expensive trial division on large inputs.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- A computer capable of running a Java development environment
- Minimum 4 GB RAM recommended
- Keyboard and terminal/command prompt

Software Requirements:

- Windows, Linux, or macOS
- Java Development Kit (JDK 17 or later)
- Apache Maven
- IDE or text editor such as IntelliJ IDEA, Eclipse, or VS Code

6. IMPLEMENTATION

1. The terminal implementation is organized into the following core modules:

   1. Main.java — Entry point. Reads the integer and iteration count, calls PrimeAnalyzer, and prints the result and measurements.

   2. PrimeAnalyzer.java — Coordinates the analysis. It runs Miller–Rabin for all inputs and runs NaivePrimeCheck only when the input bit length is within the configured small-input threshold (32 bits).

   3. MillerRabin.java — Implements the probabilistic test. It handles special cases, writes n − 1 as d × 2^s, selects random witnesses, and uses modular exponentiation and repeated squaring.

   4. NaivePrimeCheck.java — Implements deterministic trial division for smaller inputs. It checks candidate divisors up to the square root of n.

   Representative logic for the main Miller–Rabin round is shown below. The actual project source files should be used as the authoritative implementation.

```
if (n.compareTo(BigInteger.valueOf(2)) < 0) return false;
if (n.equals(BigInteger.valueOf(2)) || n.equals(BigInteger.valueOf(3))) return true;
if (!n.testBit(0)) return false;
// Write n - 1 = d * 2^s, then test random witnesses.
BigInteger x = a.modPow(d, n);
if (x.equals(BigInteger.ONE) || x.equals(n.subtract(BigInteger.ONE))) continue;
// Otherwise repeatedly square x modulo n; a failed round proves composite.
```

7. RESULTS & DISCUSSION

The following sample outputs were observed during terminal testing. Add screenshots of the terminal run before final submission if required by the instructor.

Test 1 — Input: 97; iterations: 10. Traditional result: PRIME. Miller–Rabin result: PROBABLY PRIME. The program displayed the iteration count, execution time, error bound, and confidence.

Test 2 — Input: 100; iterations: 10. Traditional result: COMPOSITE. Miller–Rabin result: COMPOSITE. This confirms that the algorithm identifies this composite input.

Test 3 — Input: 104729; iterations: 10. Traditional result: PRIME. Miller–Rabin result: PROBABLY PRIME. The two methods agreed for this small enough input.

For 10 iterations, the standard upper bound is (1/4)^10 ≈ 9.5367431641 × 10^-7, giving a confidence lower bound of approximately 99.9999046326%. This is a probabilistic bound, not a guarantee that every reported probably-prime value is mathematically proven prime.

Large-input handling: PrimeAnalyzer skips trial division when the input bit length exceeds 32 bits and uses Miller–Rabin only. Verify and capture the large-input run on the target machine before claiming a specific large-number result.

8. CONCLUSION & FUTURE SCOPE

PrimeTest demonstrates how data structures and algorithmic techniques can be applied to primality testing. The project uses BigInteger to support large integer values and Miller–Rabin to test them efficiently through modular exponentiation and randomized witnesses. For small inputs, deterministic trial division provides a baseline comparison. The terminal application reports primality status, execution time, iteration count, error bound, and confidence estimate. The current implementation meets the core goal of providing an understandable command-line tool for primality analysis. Large-input performance and output should be verified on the final demonstration machine before submission.

Future enhancements may include a graphical or web interface, automated prime generation, benchmarking across multiple primality algorithms, parallel processing, configurable input-size limits, and additional test cases. The application could also compare Miller–Rabin with other algorithms such as Fermat testing or deterministic methods where applicable. For cryptographic use, additional safeguards and carefully specified security requirements would be necessary; this educational project should not be treated as a production cryptographic key-generation system.

9. REFERENCES

1. M. O. Rabin, “Probabilistic Algorithm for Testing Primality,” *Journal of Number Theory*, vol. 12, no. 1, pp. 128–138, 1980. https://doi.org/10.1016/0022-314X(80)90084-0
2. G. L. Miller, “Riemann’s Hypothesis and Tests for Primality,” *Journal of Computer and System Sciences*, vol. 13, no. 3, pp. 300–317, 1976. https://doi.org/10.1016/0022-0000(76)90043-8
3. A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. CRC Press, 1996. https://cacr.uwaterloo.ca/hac/
4. Oracle, “Class BigInteger — Java SE 17 API Specification.” https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigInteger.html
5. Oracle, “Class SecureRandom — Java SE 17 API Specification.” https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/security/SecureRandom.html

6. Oracle, “Java Platform, Standard Edition Documentation.” https://docs.oracle.com/en/java/javase/17/
7. Apache Maven Project, “Maven Documentation.” https://maven.apache.org/guides/
8. “Miller–Rabin Primality Test,” *Wikipedia*. https://en.wikipedia.org/wiki/Miller%E2%80%93Rabin_primality_test
9. “Trial Division,” *Wikipedia*. https://en.wikipedia.org/wiki/Trial_division
10. “Modular Exponentiation,” *Wikipedia*. https://en.wikipedia.org/wiki/Modular_exponentiation